2026 第九届传智杯 · AI 大模型创新应用挑战赛

Margin 科研助手

面向科研阅读与写作全流程的可核查 AI 助手 · 技术文档

参赛赛道	StudyHack 学习科研赛道（个人赛）
作者	niuchuanbo
代码仓库	https://github.com/niuchuan66-lab/margin-research-copilot
在线演示	https://niuchuanbo-margin-research-copilot.ms.show
文档版本	v2.0 · 2026-11-03
许可证	Apache-2.0

本文档描述当前可运行版本的真实实现与实测结果。未实现的能力在“已知降级项”与“局限”中明确列出，不做超出代码的宣称。

一、项目概述

1.1 项目定位

Margin 是一个面向科研阅读与写作全流程的 Web 端科研助手。它不是“把 PDF 丢给大模型总结”的聊天框，而是把**页码与版面坐标（bbox）作为整条链路的主线**：解析阶段产出页码、坐标和章节路径，检索阶段按这些锚点召回原文，生成阶段强制模型标注 `[n]`，前端再把 `[n]` 还原成 PDF 上的精确高亮。

系统覆盖赛题要求的六个模块：

1. PDF 管理与细粒度解析、语义去重
2. 抗幻觉 RAG 与多文推理
3. 版式感知阅读与双向联动翻译
4. 论文检索、追踪与推荐
5. 引用图谱、基石节点与综述
6. 学术写作辅助与数据可视化

1.2 要解决的三个真实痛点

痛点	常见做法的问题	本项目做法
大模型回答不可核查	只给一段总结，无法回到原文	每个事实句强制 `[n]`，引用带页码、块 id、bbox，点击即跳转并高亮
长文档与扫描件不可用	同步解析 500+ 页会超时；扫描件没有文本层	逐页混合解析 + 异步任务 + 只对无文本页 OCR
文献库版本混乱	只按文件名判断重复	基于正文 5-gram Jaccard 的语义去重 + 版本号策略

1.3 交付形态

- 前端：Vue 3 + Vite + TypeScript + Pinia，单页应用
- 后端：FastAPI + Pydantic v2，异步解析任务，SSE 流式接口
- 存储：SQLite（WAL）持久化论文、文本块、参考文献、解析任务
- 模型：任意 OpenAI 兼容接口，当前使用 DeepSeek `deepseek-chat`
- 部署：单容器 Docker，端口 7860，已部署到魔搭创空间

当前状态（2026-11-03）：39 篇组委会 PDF 全部导入并回归通过；在初版基础上完成六项后续工作——混合检索（BM25 + 稀疏重叠 + 可选向量）、段落级版本差异、工具调用 Agent（含 Python 沙箱子进程）、中英双栏同步阅读、解析 kind 标记（表格/公式/图注）、稳定性与保活；后端 79 个测试、Ruff、前端类型检查与生产构建均通过；线上环境已完成带 bbox 的问答验证。

二、需求分析与对应实现

2.1 赛题模块映射

赛题模块	核心要求	本项目实现	主要入口
模块一	批量上传、语义去重、扫描件 OCR、长文档、标题树/表格/公式/参考文献	逐页混合解析、5-gram 去重、RapidOCR、异步任务、标题栈、参考文献块、表格/公式/图注 kind 标记	`POST /papers`、`POST /papers/{id}/parse`
模块二	单篇细粒度问答、多篇对比、原文锚点	混合检索（BM25 + 重叠 + 可选向量）、邻块扩展 + 最佳窗口 + 按篇召回 + 引用校验；工具调用 Agent	`POST /qa/ask`、`POST /qa/stream`、`POST /agent/run`
模块三	版式阅读、标题树、引用与参考文献跳转、双语/划词翻译	PDF.js 渲染、1–3 级目录、参考文献面板、中英双栏同步滚动、正文 `[n]` 跳转、悬停/划词翻译	`/reader` 系列接口
模块四	arXiv/OpenAlex 检索、推荐、GitHub 复现性	arXiv Atom + OpenAlex API、关键词抽取、GitHub 链接优先	`POST /discovery/search`、`/recommend`
模块五	引用图谱、基石节点、综述、Future Work	引用匹配 + 外部节点剪枝 + PageRank + LLM 综述 + Future Work 抽取合并	`POST /graph/build` 等
模块六	无幻觉大纲、TikZ/Graphviz/Mermaid、CSV 图表与图注	文献约束的 LLM 大纲、三种图脚本模板、折线/柱状/雷达图 + LLM 图注	`POST /writing/*`

2.2 非功能需求

需求	设计
500+ 页不阻塞请求	解析立即返回 job，后台线程执行，前端轮询进度
扫描件可用	只有无文本页才走 OCR；15 页扫描件实测 161.7 秒
没有重型服务也能启动	Docling/GROBID/Qdrant/BGE 全部是可选适配器，缺失时自动降级
结果可复现	Docker 单容器、固定依赖清单、环境变量模板、启动时重建索引
密钥不进入仓库	`.env` 与 `.env.modelscope` 均被 gitignore；线上用创空间 Secret
长文档内存可控	阅读器按页拉取 blocks，不一次返回整篇

三、总体技术路线

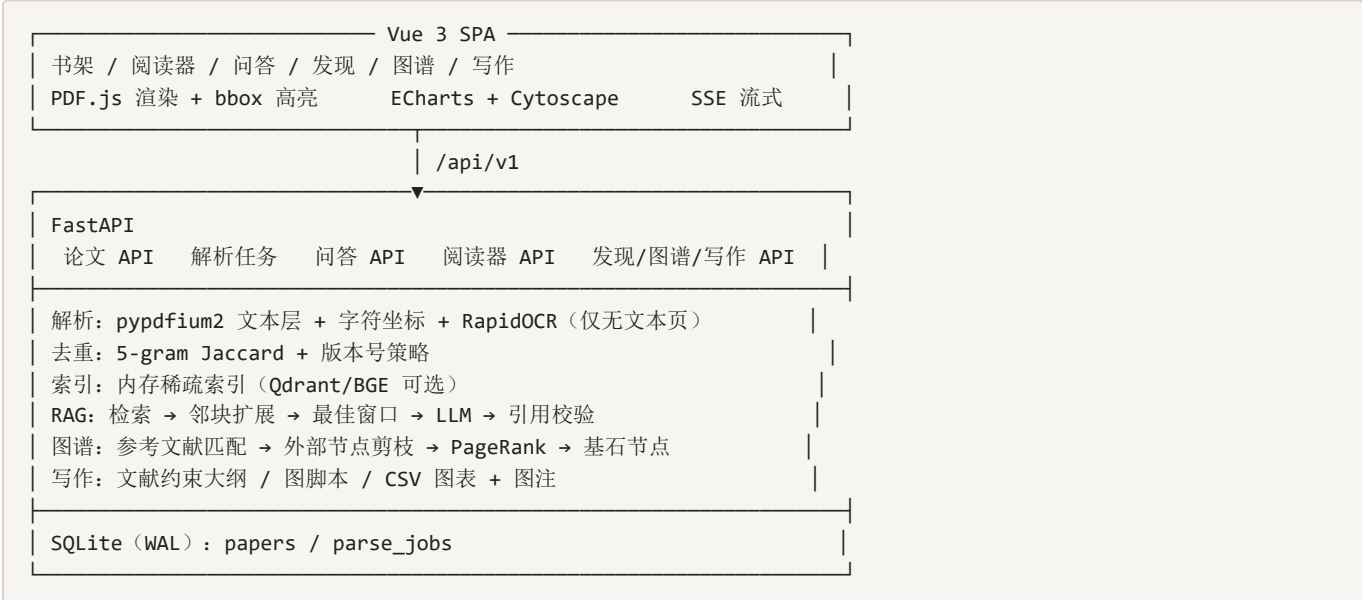
3.1 选型原则

1. **许可证干净**：主许可证 Apache-2.0，核心链路只使用 MIT / BSD / Apache-2.0 组件；MinerU、PyMuPDF、PDFMathTranslate 等 AGPL 组件不进入主进程。
2. **每一层都可降级**：任何重型依赖缺失，系统仍能启动并完成演示，不把“装不上”变成“跑不了”。
3. **锚点优先**：解析层必须产出页码与 bbox，否则后续 RAG、引用跳转、PDF 高亮都无从谈起。
4. **本地可复现优先**：默认 SQLite、内存索引、本地 OCR，不要求评审环境先起 PostgreSQL/Qdrant/GROBID。

3.2 技术栈

层	选型	作用
前端框架	Vue 3 + Vite + TypeScript + Pinia + Vue Router	六个模块页面与状态管理
PDF 渲染	PDF.js	单双栏渲染、页码、bbox 高亮
图表/图谱	ECharts、Cytoscape.js	引用网络、折线/柱状/雷达图
公式/拓扑	KaTeX、Mermaid	公式与架构图
API	FastAPI + Uvicorn + Pydantic v2	异步接口、SSE、数据契约
Agent 编排	规则意图路由 + LLM Planner + 工具注册表执行器	plan → tool_start/tool_result → answer → citations
检索	纯 Python BM25 + 关键词重叠 + 可选 embedding 向量腿	无需外部服务即可混合打分
PDF 解析	pypdfium2	文本层、字符坐标、页面栅格
OCR	RapidOCR（ONNX Runtime），PaddleOCR 可选	扫描件识别
分词修复	wordninja	修复 OCR 丢失空格
图论	NetworkX；缺 SciPy 时纯 Python PageRank	基石节点与中心度
持久化	SQLite（WAL）	论文、任务、进度
模型	DeepSeek `deepseek-chat`（OpenAI 兼容）	问答、综述、大纲、翻译、图注
部署	Docker 多阶段构建 + 魔搭创空间	单容器、7860 端口、持久化目录

3.3 架构与数据流



一条典型请求的数据流：

```
PDF
 → 页级文本/OCR
 → Block(page, bbox, kind, section_path, text)
 → Chunk(paper_id, page, block_id, bbox, section_path, text)
 → 检索与邻块扩展
 → LLM 回答（带 [n]）
 → Citation(page, block_id, bbox, quote)
 → 前端跳页并绘制 PDF 高亮
```

3.4 许可证链路

主许可证 Apache-2.0。核心依赖均为 MIT / BSD / Apache-2.0：Vue、Vite、Pinia、FastAPI、Pydantic、PDF.js、ECharts、KaTeX、Mermaid、LangGraph、pypdfium2、RapidOCR、NetworkX、SQLite。AGPL 组件（MinerU、PyMuPDF、PDFMathTranslate）不进入主进程；第三方声明见 `THIRD_PARTY_NOTICES.md`。

3.5 AI 技术方案：选型理由、集成方式与优化策略

（1）选型理由

决策	选择	理由
生成模型	OpenAI 兼容接口，默认 DeepSeek `deepseek-chat`	中文与长上下文表现稳定、成本低、无需自托管；接口层只依赖 `/chat/completions`，换模型只改环境变量
微调	不做微调	科研问答的事实来源是用户库内原文，靠检索与引用约束解决，微调反而增加幻觉与维护成本
编排	规则路由优先，LangGraph / LLM Planner 兜底	常见意图零延迟命中；未知意图才付出一次规划调用，且规划结果受注册表校验
检索	进程内 BM25 + 关键词重叠，向量腿可选	评审环境零外部服务即可运行；配置 embedding 端点后无需改接口即可增强同义召回
解析	pypdfium2 文本层 + 字符坐标，RapidOCR 仅补无文本页	许可证干净、逐页可控；OCR 是成本最高的一步，只对必要页执行
代码执行	AST 白名单 + `python -I` 子进程	模型生成的代码只作为不可信输入处理，绝不进入 API 进程

（2）集成方式

```
用户问题
 → 意图路由（规则 → LLM Planner → 默认 library.answer）
 → 工具执行（检索 / Agent / 图谱 / 写作 / Python 沙箱）
 → Prompt 组装（<context> + 引用契约 + 输出标记）
 → LLM（temperature=0.1，max_tokens=2000）
 → 引用校验 → SSE 推送 answer / citations
```

- 模型调用集中在 `services/llm.py`，统一超时、温度与失败返回，业务层用 `None` 判定降级；
- 提示词集中在 `agent/prompts.py`，系统提示强制“只用上下文、每个事实句带 `[n]`、缺失写 `[未明确给出]`”；
- 结构化输出（大纲、规划、修订总结）要求严格 JSON，解析失败或字段不足即回退模板/确定性结果；
- 所有生成结果都带 `model` 字段，未配置模型时明确标注抽取式降级，不冒充模型输出。

（3）优化策略

- **召回**：BM25 与重叠分归一融合；多篇提问按论文分别召回再交错合并；命中块前后各扩 1 块，再用最佳窗口截取；

- **可信**：引用必须带页码、block id 与 bbox；verify 节点校验 quote 是否出现在检索文本中；`grounded` 要求回答中的 `[n]` 至少对应一条已校验引用；
- **成本**：单次问答只调用一次生成模型；翻译按 block 批量、按文本哈希缓存；检索与解析不调用模型；
- **时延**：解析走后台任务并按并发上限排队；问答 SSE 先算后分片，保证答案与引用一致；健康检查的 LLM 探测单独限时 4 秒；
- **稳定性**：LLM 调用失败时逐级降级到抽取式回答、模板或明确报错，不让单次外部故障拖垮接口。

四、关键实现：每一层是怎么做出来的

4.1 解析流水线：把“页码 + 坐标”做成一等公民

文件：`backend/app/services/parsing.py`

（1）逐页混合解析，而不是整篇硬解

`parse_pdf` 逐页读取文本层：

- 有文本层：直接走 pypdfium2，不跑 OCR；
- 无文本层：记为 OCR 候选页；
- 全部页处理完后，只对候选页加载 RapidOCR。

这样 541 页的 UDL 只有 4 页无文本，OCR 成本被限制在 4 页；15 页纯扫描件才会 15 页全跑 OCR。

（2）bbox 不是“整页框”，而是字符框并集

pypdfium2 的 `get_charbox(i)` 能返回每个字符的 PDF 坐标。解析时先按行记录字符偏移，再对每个段落/标题的字符区间做并集，得到 `[left, bottom, right, top]`，坐标空间统一为 PDF 的 bottom-left。

这一步解决了原版本的致命问题：文本层 PDF 的 101 个块 bbox 全为空，导致前端只能跳页、不能高亮。修复后 v1 的 101/101 个块都有 bbox。

（3）按行重建阅读顺序，而不是“先段落再标题”

原实现先把整页切成若干段，再额外追加标题块，导致：

- 标题重复出现在正文里；
- 块顺序与阅读顺序不一致；
- 标题无法影响后续段落的章节路径。

新实现逐行扫描：

```
遇到标题 → flush 当前段落 → 更新章节栈 → 输出 heading 块
遇到正文 → 累积到段落缓冲
遇到 References → 切换参考文献模式，按 [n] 分组输出 reference 块
页尾 → flush 段落与参考文献
```

章节栈按编号深度维护：`1` 为一级，`3.2` 为二级，`3.2.1` 为三级；栈跨页保留，所以第 2 页开头的正文能继承第 1 页的 `1 Introduction`。

（4）三类“假标题”过滤

真实 PDF 里会混入目录行、页眉、版权行，它们都长得像标题：

- **目录行**：一页中出现 3 行以上点引导线（`....`）时，整页按目录处理，不更新章节栈；

- **页眉**：形如 `1.1 Supervised learning 3`（结尾是页码）的行不当作标题；
- **版权/期刊头**：以 `Journal of`、`Published in`、`Proceedings`、`Association for Computational Linguistics` 等开头的行不当作标题。

UDL 的标题树因此从“627 个混杂标题”收敛为 442 个 1–3 级节点。

（5）参考文献变成可跳转的块

进入 References 后，以 `[n]` 为分隔符把条目分组，输出 `kind=reference` 的块，并保留页码与 bbox。阅读器的参考文献面板直接消费这些块，点击即可跳到对应页并高亮条目。

（6）标题提取：元数据优先，版式兜底

- 先读 PDF 元数据 `Title`（BERT 的 PDF 就带正确标题）；
- 没有元数据时，扫描首页前 6 个块的行，跳过版权/期刊/作者行，取第一条标题行；
- 标题跨行时做拼接：以 `:` 结尾、全大写续行、或下一行以小写开头且不超过 6 个词。

修复后 39 篇里绝大多数标题恢复为真实论文名，而不是 `2005.14165v4` 这样的文件名。

（7）OCR 坐标转换

RapidOCR 返回的是 `scale=2` 渲染图上的左上角像素坐标。前端 PDF.js 需要的是 PDF 点坐标、bottom-left。转换公式：

```
x_pdf = x_pixel / scale
y_pdf = page_height - y_pixel / scale
```

不做这一步，扫描件的高亮会偏移数百像素。

（8）长文档的异步化

文件：`backend/app/services/jobs.py`、`backend/app/api/v1/papers.py`

`POST /papers/{id}/parse` 立即创建 job 并返回，实际解析放在 `asyncio.to_thread` 中执行；进度写入 SQLite 的 `parse_jobs` 表，前端轮询 `parse-status`。服务重启时，处于 `queued/running` 的任务会被标记为失败，避免前端永远转圈。

4.2 语义去重：看正文，不看文件名

文件：`backend/app/services/dedup.py`

1. 取每篇论文前 200 个文本块，归一化后切成 5-gram；
2. 计算 Jaccard 相似度；
3. `>= 0.62` 判为重复；`0.45–0.62` 给出人工确认警告；
4. 从文件名/标题提取 `vN`，较新版本建议 `replace`，较旧版本建议 `keep`。

实测：`1706.03762v1` 与 `v7` 相似度 **0.7174**；先传 v7 再传 v1 时，系统建议保留 v7；先传 v1 再传 v7 时建议覆盖。前端在解析完成后弹出“用新版覆盖 / 保留两者 / 稍后处理”三个操作。

4.3 抗幻觉 RAG：从“有引用”到“引用可验证”

文件：`backend/app/services/qa.py`、`backend/app/agent/graph.py`、`backend/app/agent/prompts.py`

（1）检索

默认使用进程内混合索引，三条腿并行打分后 min-max 归一合并：

- **BM25**：纯 Python 实现（k1=1.5，b=0.75），英文词 + 中文二元组切分，增量维护 DF 与平均文档长度；
- **关键词重叠**：命中 token 数 / sqrt(块长度)，并给完整短语命中加 0.5；
- **向量腿（可选）**：配置 `EMBEDDING_BASE_URL` 后调用 OpenAI 兼容 `/embeddings`，余弦相似度参与加权；未配置或调用失败时自动关闭该腿。

无向量腿时权重为 BM25 0.6 / 重叠 0.4；有向量腿时为 0.5 / 0.3 / 0.2。Qdrant + BGE-M3 仍是可选适配器，服务存在时整体切换到 Qdrant dense 检索。

（2）邻块扩展与最佳窗口

检索命中的块会向前后各扩 1 块，保证跨行/跨页的上下文完整；送给模型前用 `best_window` 截取问题相关窗口，而不是整块硬塞。

（3）多篇论文按篇召回

原实现是全局 top-k，一篇论文可能占满上下文。现在多篇提问时：

```
对每篇论文分别检索 top-k
 → 按名次交错合并
 → 再统一做邻块扩展
```

实测 GPT-1 / GPT-3 / BERT / Transformer 四篇联合提问时，引用能覆盖全部四篇。

（4）Prompt 约束

系统提示词强制：

- 只使用 `<context>`；每个事实句带 `[n]`；
- 原文直接表述标 `[原文声明]`，模型归纳标 `[模型归纳]`；
- 缺失字段写 `[未明确给出]`，歧义写 `[不确定]`；
- 找不到依据时直接说“库中未找到足够依据”，不猜测。

（5）回答后的引用校验

- 每条 Citation 都带 `page`、`block_id`、`bbox`、`quote`；
- LangGraph 的 verify 节点检查 quote 是否真的出现在检索文本中；
- `grounded` 不再等于“有引用”，而是要求回答里的 `[n]` 至少有一个能对应到校验后的引用列表；
- 前端引用 chip 点击后跳页并绘制 bbox 高亮。

实测单篇问答：β1=0.9、β2=0.98、ε=1e-9、28.4 BLEU、8×P100/3.5 天，全部带页码与 bbox。对于 GPT-1 原文没有写参数量的问题，系统明确回答“库中未找到足够依据”，而不是编造 117M。

4.4 阅读器：目录、参考文献、双栏同步与翻译

文件：`frontend/src/views/ReaderView.vue`、`frontend/src/components/BilingualPane.vue`、`backend/app/api/v1/reader.py`

- **目录**：`GET /reader/{id}/outline` 返回标题、页码、1–3 级层级；点击跳页。
- **参考文献**：`GET /reader/{id}/references` 返回编号、页码、bbox、原文；点击跳页并高亮。
- **中英双栏**：PDF 在左、双语文本在右（<1100px 回退上下布局）；右侧按页分组，翻页自动定位到对应分组，手动滚动右侧分组时经 `IntersectionObserver` 反驱 PDF 翻页（带 500ms 抑制窗口防回环）。
- **正文 `[n]`**：双语原文中的 `[数字]` 渲染为按钮，点击滚动参考文献列表并闪烁高亮对应条目；条目带 bbox 时同步跳转 PDF 页并高亮。

- **悬停翻译**：鼠标在未翻译原文块停留 500ms 出 popover 译文（`translate-text`），结果按文本哈希缓存，同一段只请求一次；失败可重试。
- **整页 / 局部翻译**：`POST /reader/translate` 按 block id 批量翻译并跨页累积；`POST /reader/translate-text` 处理选中/粘贴文本，保留公式、变量、引用标记。
- **PDF 渲染**：PDF.js 按页渲染，`convertToViewportRectangle` 把 PDF 坐标转换成画布坐标，叠一层高亮 div。

4.5 发现与推荐

文件：`backend/app/services/discovery.py`

- arXiv：Atom API，解析标题、作者、年份、摘要、DOI、arXiv id；从摘要正则提取 GitHub 链接。
- OpenAlex：JSON API，重建倒排摘要，同样提取 GitHub 链接。
- 推荐：从库中标题抽取高频关键词（去停用词），检索 arXiv；过滤已在库中的标题；**带 GitHub 链接的论文排在前面**，并在推荐理由中标注“含 GitHub 复现代码”。

4.6 图谱、综述与未来方向

文件：`backend/app/services/graph_service.py`、`backend/app/api/v1/graph.py`

（1）引用边

对每篇论文的参考文献，按 DOI → 标题精确匹配 → 标题包含 → 原始条目包含 的顺序匹配库内论文。匹配不到的外部文献生成 `ref:` 节点。

（2）外部节点剪枝

早期版本把每一条参考文献都建成节点，39 篇论文产生上千个节点，前端几乎不可读。现在只保留**被 2 篇及以上库内论文引用**的外部文献；一次性引用不再上图。

（3）中心度

优先 `networkx.pagerank`；如果运行环境缺 SciPy 导致 PageRank 不可用，则回退到 40 次迭代的纯 Python PageRank。39 篇实测 361 节点 / 803 条边，计算约 3 秒。

（4）综述

`POST /graph/{id}/survey` 把核心节点、引用边、摘要片段交给 LLM，要求只依据给定材料、每个论断用 `[论文标题]` 标注来源、证据不足时明确写“证据不足”。没有 LLM 时回退到模板。

（5）Future Work

`POST /graph/{id}/future-work` 在各论文的标题中查找 `Future Work / Future Direction / Open Problems`，取其后 1–3 个正文块作为证据；按 5-gram Jaccard ≥ 0.7 合并相似方向；按年份倒序返回。39 篇实测抽取 10 条。

4.7 写作与可视化

文件：`backend/app/services/writing.py`、`backend/app/api/v1/writing.py`

- **大纲**：把 idea 与库内前 10 篇论文的标题/摘要交给 LLM，要求输出严格 JSON（标题 + 6–8 节 + 每节引用的文献编号）；解析失败或章节不足 4 节时回退到模板。References 只来自库内论文与已解析参考文献，不编造。
- **图脚本**：Mermaid / Graphviz / TikZ 三种模板；输入本身是合法脚本时直接透传。
- **CSV 图表**：自动识别数值列与类别列，支持折线、柱状、雷达三种；雷达图把每个数值列作为一个指标、每行作为一个样本；图注由 LLM 根据前 10 行数据生成，只描述可见趋势，不编造显著性。

4.8 持久化、启动恢复与部署

文件：`backend/app/store.py`、`backend/app/main.py`、`Dockerfile`

- SQLite 使用 WAL，`busy_timeout=5000`；
- `papers` 表存论文 JSON（含 blocks/references/metadata），`parse_jobs` 表存任务；
- 启动时加载论文、把中断的 `parsing/running` 标记失败、对 ready 论文重建检索索引；
- 正常关闭时 checkpoint WAL；
- Docker 多阶段：Node 22 构建前端 → Python 3.12 运行后端；端口 7860；`FRONTEND_DIST=/app/frontend_dist`；数据目录 `/mnt/workspace/data`；
- 魔搭创空间用 Secret 注入 `LLM_API_KEY`，普通变量配置 `LLM_BASE_URL` / `LLM_MODEL`。

4.9 工具调用 Agent：从问答到任务执行

文件：`backend/app/agent/`（router / executor / tools / graph）、`frontend/src/stores/chat.ts`

（1）意图路由

规则优先解析常见请求（arXiv 检索/下载、CSV 出图、版本对比、大纲、综述、未来方向、图谱、Python 计算），命中直接生成计划；未命中时由 LLM Planner 输出严格 JSON 计划（最多 3 步，工具名必须在注册表内）；Planner 也失败则回退到库内问答 `library.answer`，保证任何输入都有可用回答。

（2）工具注册表（10 个）

`library.search` / `library.answer`、`arxiv.search` / `arxiv.download`、`graph.build` / `graph.survey` / `graph.future_work`、`writing.outline` / `writing.chart`、`python.exec`。每个工具声明 Pydantic 入参模型，执行器负责校验与错误透传。

（3）Python 沙箱（安全边界）

用户代码永不进入 API 进程：AST 白名单只放行 `math/statistics/csv/json/io/re` 与安全内置函数，拒绝 `open/eval/exec/import/下划线属性/装饰器`；通过独立子进程 `python -I` 执行，8 秒超时、8KB 输出截断。工具结果摘要携带 stdout（截断 400 字符），LLM 才能基于真实输出汇总答案。

（4）SSE 过程与前端

`POST /api/v1/agent/run` 按 `plan → tool_start / tool_result* → answer → citations → done` 推送事件；前端 Agent 模式实时把工具名、参数摘要、结果摘要写入过程面板，回答区展示引用与「去书架查看」等 artifact 入口。

4.10 段落级版本差异与修订总结

文件：`backend/app/services/version_diff.py`、`backend/app/api/v1/papers.py`

- **段落对齐**：对 v1/v7 的块文本做 5-gram shingle 与倒排索引，按相似度贪心配对（<0.35 新增、0.35–0.85 改写、已配对未匹配为删除，≥0.85 视为未变化不展示）；
- **LLM 修订总结**：差异清单交给 LLM 生成中文总结，要求引用 `[新版 p.X]` / `[旧版 p.Y]` 页码标签，服务端用正则校验标签合法性；无 LLM 时回退确定性统计；
- **实测**：v1/v7 整体相似度 0.7174，47 处变化（25 改写 / 14 新增 / 8 删除），每条都带双方页码锚点。

4.11 解析增强：表格 / 公式 / 图注 kind 标记

文件：`backend/app/services/parsing.py`

只在 `Block.kind` 上做低风险标记，不改数据契约：

- **表格**：文本层按字符包围盒找 ≥12pt 的横向间隔作为列边界，连续 ≥2 行列边界对齐（±6pt）时合并为一个 `table` 块并保留原始行文本；两列行要求右侧单元格以数字/公式开头，避免正文换行误判；
- **公式**：`=` 或数学符号占比高、长英文词不超过 8 个的行标记 `formula`；
- **图注**：`Figure/Fig./Table N` 后接冒号、句点或大写单词的行标记 `figure_caption`（`Table 2 summarizes...` 这类正文不误判）；
- **OCR 路径**按行复用同一套图注/公式提示；解析结果 `metadata.block_kinds` 记录各 kind 数量；
- **前端**：双语右侧按 kind 渲染（表格橙色左边条 + 等宽字体 + 保留换行，公式紫色，图注青色）并显示角标。

实测 `1706.03762v1.pdf`：139 块 = heading 24 / paragraph 52 / table 7 / formula 11 / figure_caption 6 / reference 39，139/139 带 bbox；金标检索无回归。

4.12 稳定性、性能与保活

文件：`backend/app/api/v1/papers.py`、`backend/app/main.py`、`backend/app/api/v1/health.py`、`backend/scripts/keepalive.py`、`backend/scripts/stress.py`

- **上传校验**：后缀 → MIME → 前 1024 字节 `%PDF` 文件头 → 大小（`MAX_UPLOAD_MB`）→ 页数（`MAX_PDF_PAGES`，失败即删临时文件），分别返回 400/413；
- **解析并发**：`asyncio.Semaphore(PARSE_CONCURRENCY=2)`，超出的任务先置为 `queued`（“排队等待解析”），拿到信号量后转 `running`；
- **统一异常处理**：每个请求分配 `request_id` 并回写 `X-Request-ID`；未捕获异常返回 `500 {"detail": "服务器内部错误", "request_id": ...}`，日志用同一 rid 记录路径、状态码与耗时；
- **健康深探测**：`/health` 返回 `index_chunks`、`llm_latency_ms`（1-token 往返，4s 上限）、`memory_rss_mb`（Windows psapi / POSIX statm）；
- **保活与压测**：`keepalive.py`（纯标准库，单次探测退出码 0/1，连续 3 次失败退出码 2）可直接挂 Windows 任务计划；`stress.py`（异步 httpx）并发上传解析并跑问答，输出 P95 与 RSS 差值，结束后自动清理。

实测：5 个并发解析全部成功（2 个直接运行、3 个排队后运行，墙钟 168.6 秒）；20 并发问答 P95 26.3 秒；进程 RSS 69.3 → 217.2 MB。

五、实施过程：这个版本是怎么一步步做出来的

5.1 里程碑

阶段	产出	遇到的问题	解决方式
1. 赛题拆解	六大模块、评分权重、许可证约束	官方页面抓取受限	用 curl/API 拉取原文，整理为本地赛题文档
2. 骨架	Vue 六页面 + FastAPI 六组接口 + Pydantic 契约	依赖多、首次安装慢	接口先定契约，重型能力全部做可选适配器
3. 本地跑通	前后端启动、DeepSeek 连通	Python 安装器报错、embed 包无法 `-m`	改用 embeddable Python，补 `check_llm.py` / `sys.path`
4. 扫描件	RapidOCR 接入、空格修复	OCR 输出粘连（`AttentionIsAll YouNeed`）	wordninja 修复，并保护人名、邮箱、全大写缩写

阶段	产出	遇到的问题	解决方式
5. 长文档	异步解析 + 进度轮询 + 分页加载	541 页同步解析阻塞请求	job + SQLite 进度 + 前端轮询恢复
6. 去重	5-gram 去重 + 版本策略 + 前端三操作	v1/v7 相似度 0.6894，低于原阈值 0.72	阈值调到 0.62，并把文件名版本号纳入判断
7. 持久化	SQLite papers/jobs、重启恢复	重启后任务卡在 parsing	启动时把中断任务标记失败，ready 论文重建索引
8. 部署	Docker + 魔搭创空间 + Secret	平台不注入账号级托管 Key	改用创空间 Secret `LLM_API_KEY`，重新部署验证
9. 官方回归	39 篇 PDF、六模块实测	bbox/章节路径为空、多篇检索覆盖不足、图谱噪声大	见 5.2

5.2 官方测试集回归中发现并修复的问题

这一轮不是“再写点功能”，而是拿组委会 39 篇 PDF 真跑，按结果反推问题。

问题一：文本层 PDF 的 bbox 全为空

- 现象：v1 的 71 个块 bbox 全是 `None`，前端只能跳页，不能高亮。
- 原因：旧解析只取整页文本，没有记录字符坐标。
- 修复：逐行记录字符偏移，按块做字符框并集；坐标统一为 PDF bottom-left。
- 结果：v1 101/101 块带 bbox。

问题二：章节路径全为空，目录层级全是 1

- 原因：旧实现先切段落、再单独追加标题，标题不影响段落，且 `section_path` 从未赋值。
- 修复：按行重建阅读顺序，维护跨页章节栈；标题编号决定入栈深度。
- 结果：v1 目录出现 1/2/3 级；UDL 442 个标题节点层级正确。

问题三：扫描件高亮错位

- 原因：OCR 返回图像像素坐标，前端按 PDF 点坐标解释。
- 修复：除以渲染 scale，并按页高翻转 y。
- 结果：15 页扫描件 955 个块全部可定位。

问题四：多篇论文对比时只答一篇

- 现象：问 GPT-1/GPT-3/BERT/Transformer 四篇，模型说“库中未找到 GPT-1 信息”。
- 原因：全局 top-k 被其中一篇占满。
- 修复：按论文分别召回，再按名次交错合并。
- 结果：引用覆盖全部四篇；对原文确实没有的参数量，模型明确拒绝编造。

问题五：论文标题是文件名

- 原因：旧标题启发式要求“首字母大写词占多数”，`Mistral 7B`、`GPT-4` 这类含数字的标题被判失败。
- 修复：PDF 元数据优先；版式兜底改为跳过版权/期刊/作者行，允许句子式标题，支持跨行拼接。
- 结果：BERT、Mistral、InstructGPT、T5、LLaMA 等标题恢复。

问题六：图谱节点爆炸

- 现象：39 篇产生上千节点，画布不可读。
- 修复：只保留被 2 篇以上论文引用的外部文献；参考文献标题从整条条目改为启发式抽取。
- 结果：361 节点 / 803 边，基石节点都是真实论文名。

问题七：写作与综述是模板桩

- 修复：综述改为“图谱 + 摘要 + 证据约束”的 LLM 生成；大纲改为文献约束的结构化 JSON；Future Work 新增抽取与合并；图表增加柱状/雷达与 LLM 图注。
- 结果：大纲 7 节、综述 6845 字符、Future Work 10 条、三种图表均可用。

问题八：GitHub 推送失败

- 现象：`Failed to connect to github.com port 443`，但系统代理是开着的。
- 原因：Git for Windows 使用 libcurl，不读取 Windows 系统代理。
- 修复：对 GitHub 单独配置 `http.https://github.com.proxy`，或命令级 `-c http.proxy=...`；推送成功。

5.3 关键工程取舍

取舍	选择	理由
解析引擎	pypdfium2 + RapidOCR，而不是默认上 Docling/MinerU	许可证干净、安装轻、逐页可控；Docling 仍保留为可选适配器
向量库	默认内存稀疏索引，Qdrant/BGE 可选	评审环境不需要先起外部服务；接口不变，可平滑替换
前端渲染	PDF.js 画布 + 绝对定位高亮	避免引入 PyMuPDF 等 AGPL 组件，同时能精确还原 bbox
异步任务	进程内线程 + SQLite，而不是 Celery/Redis	单容器部署简单；重启恢复逻辑已覆盖
长文档	按页拉 blocks，不一次返回整篇	541 页整篇 JSON 会拖垮浏览器
流式	先算完再分片推送 SSE	保证引用与答案一致；真正 token 级流式列为后续优化

六、测试与实测结果

6.1 工程检查

```
cd backend && pytest          # 79 passed
cd backend && ruff check .     # All checks passed
cd frontend && npm run typecheck
cd frontend && npm run build
```

测试覆盖：健康检查与稳定性（上传校验、并发上限、request_id）、去重、写作/图表、解析辅助函数与 kind 提示、索引与混合检索、版本差异、Agent 路由/执行器/Python 沙箱、存储。

6.2 官方 39 篇回归

用例	结果
批量导入	39/39 解析成功（38 篇文本层 + 1 篇纯扫描）
v1 / v7 去重	相似度 0.7174；新版建议 `replace`，旧版建议 `keep`

用例	结果
扫描件 `1706.03762v1_img.pdf`	15/15 页、955 块、161.7 秒，全部带 bbox
文本层 `1706.03762v1.pdf`	139 块（7 表格 / 11 公式 / 6 图注）；139/139 带 bbox
541 页 UDL	2120 块、约 14 秒；537 页文本层 + 4 页 OCR；442 个标题节点
混合检索金标	21/21 页命中（本地论文复验 14/14，与旧重叠检索对比提升）
v1/v7 版本差异	47 处变化（25 改写 / 14 新增 / 8 删除），LLM 总结带页码标签
Agent 工具调用	12 类意图用例通过；python.exec 沙箱端到端回答“平均值是 6”
中英双栏阅读	翻页自动定位、`[n]` 跳转并闪烁参考文献、悬停翻译命中缓存（2 次悬停仅 1 次请求）
稳定性压测	5 并发解析（并发上限 2，排队 3）、20 并发问答 P95 26.3s、RSS +147.9 MB
单篇问答	β1/β2/ε、28.4 BLEU、8×P100/3.5 天，均带页码与 bbox
多篇问答	GPT-1 / GPT-3 / BERT / Transformer 四篇联合检索，覆盖全部来源
引用图谱	39 篇 → 361 节点 / 803 条边；PageRank 约 3 秒
综述 / Future Work	6845 字符；10 条方向并按相似度合并
写作	7 节大纲；折线/柱状/雷达图 + LLM 图注
检索推荐	arXiv / OpenAlex 各 5 条；推荐优先带 GitHub 链接

6.3 线上验证

魔搭创空间使用同一套 Dockerfile 构建并运行。初版部署已完成以下验证：

- `/api/v1/health`：`llm=true`、`rapidocr=true`、`langgraph=true`
- 线上 v1 重新解析后：101/101 块带 bbox，39 条参考文献可跳转，目录 1–3 级（初版解析器）
- 线上问答：`model=deepseek-chat`、`grounded=true`、引用 11/11 带 bbox
- 线上图谱 / 综述 / 雷达图 / 大纲均验证通过

本轮新增能力（混合检索、版本差异、工具调用 Agent、双语双栏、kind 标记、稳定性与保活）已在本地与同镜像构建中验证；提交前按同一 Dockerfile 重新部署，并复验 `/api/v1/health` 的 `index_chunks`、`llm_latency_ms`、`memory_rss_mb` 三个新字段，确保线上版本与本文一致。

6.4 已知降级项

能力	当前状态	影响
Docling	未安装	表格只做块级标记，不还原单元格结构；公式不转 LaTeX
GROBID	未运行	参考文献用正则回退，作者/年份可能不全
Qdrant + BGE-M3	未部署	默认 BM25 + 重叠混检；配置 `EMBEDDING_BASE_URL` 可加向量腿
Reranker	未接入	不做二次重排
真正的 token 流式	未实现	SSE 是算完后分片，不是模型增量输出
双栏块级同步（隐藏浏览器）	逻辑已实现	自动化环境无法触发 IntersectionObserver，需在真实可见浏览器演示
用户体系	未实现	单机单用户，无登录与多租户

七、创新点

1. **页码 + bbox 全链路锚点**：解析产出字符级坐标，RAG 引用带页码与 bbox，前端直接高亮；不是“给个链接”，而是“指到原文那一行”。
2. **引用可验证的 grounded**：回答中的 `[n]` 必须能对应校验后的引用列表，找不到依据就明确拒答；把“抗幻觉”从口号变成可测逻辑。
3. **按篇召回的多文推理**：多篇提问时每篇独立召回再交错合并，避免单一论文占满上下文。
4. **内容级去重 + 版本策略**：5-gram Jaccard 识别 v1/v7，结合版本号给出覆盖/保留建议，不依赖文件名。
5. **Future Work 抽取与合并**：从各论文的 Future Work 标题下抽证据，按相似度合并重复方向，按年份排序。
6. **全链路可降级**：没有 Docling/GROBID/Qdrant/BGE/LLM 时仍能启动，分别回退到 pypdfium2/RapidOCR/正则/内存索引/抽取式回答。
7. **单容器可复现部署**：前端构建与后端运行时打进同一个镜像，7860 端口，数据目录挂载 `/mnt/workspace`，Secret 注入密钥。
8. **工具调用 Agent 与安全沙箱**：规则优先 + LLM Planner 回退生成计划，10 个工具注册表统一校验；Python 计算在 AST 白名单 + 独立子进程 + 超时截断的安全边界内执行。
9. **版本差异可读化**：段落级对齐给出新增/改写/删除，LLM 修订总结的页码标签经服务端正则校验，不合法即回退确定性总结。
10. **零服务混合检索**：BM25 + 关键词重叠 + 可选向量腿在进程内完成归一融合，评审环境无需先起向量库；配置 embedding 端点即可平滑升级。
11. **面向演示的可观测性**：`request_id` 贯穿日志与错误响应，健康检查带索引块数/LLM 时延/进程内存，附保活与压测脚本，线上可用性可被持续验证。

八、局限与下一步

8.1 当前局限

- 默认检索是 BM25 + 关键词重叠；向量腿未部署，同义改写召回的提升需要配置 embedding 端点后才生效。
- 表格、公式、图注已做块级识别，但没有单元格结构与 LaTeX 还原。
- Agent 计划最多 3 步，工具结果不做跨步引用校验；复杂多轮任务仍需要人工收敛。
- 翻译是页级/块级批处理，不是整篇一次性翻译。
- SQLite 单机，无并发写入与多用户隔离。
- 免费创空间实例可能被回收，评审期可用性需要保活或备用部署。

8.2 下一步优先级

优先级	事项	状态
P0	演示视频、技术文档 PDF、README 团队信息	进行中（本文档即技术文档源）
P1	工具调用循环、受限 Python 执行、按意图路由	已完成（10 工具 + AST 沙箱）
P2	v1/v7 版本差异总结（新增/删除/改写 + 双方页码）	已完成（47 处变化实测）
P3	中英双栏同步滚动、正文 `[n]` 点击、划词翻译	已完成（悬停 500ms + 缓存）
P4	混合检索与重排、表格/公式结构化、整篇翻译	混检已完成；重排与结构化待做
P5	线上保活与备用部署	保活脚本已完成；备用部署待做

九、附录

9.1 目录结构

```
backend/
  app/
    api/v1/          papers / qa / reader / discovery / graph / writing /
                     versions / agent / health
    agent/           意图路由 router、计划执行 executor、LangGraph 状态机、提示词
      tools/         library / arxiv / graph / writing / python_exec（工具注册表）
    services/        parsing / dedup / indexing / qa / translation / discovery /
                     graph_service / writing / version_diff / llm / jobs
    models/          Pydantic 数据契约
    store.py         SQLite 论文仓库
  scripts/           keepalive / stress / eval_golden
  tests/             解析 / 去重 / 索引 / 问答 / 版本对比 / Agent / 沙箱 / 稳定性
frontend/
  src/
    views/           书架 / 阅读 / 问答 / 发现 / 图谱 / 写作 / 版本对比
    components/      PDF 阅读器、双语面板、Agent 过程、图谱、图表、引用、Mermaid
    stores/          Pinia
    api/             REST + SSE
deploy/
  requirements.txt
Dockerfile
docker-compose.yml
.env.example
```

9.2 主要环境变量

变量	说明	默认
`LLM_BASE_URL`	OpenAI 兼容接口地址	空
`LLM_API_KEY`	模型密钥（线上用 Secret）	空
`LLM_MODEL`	模型名	`deepseek-chat`
`EMBEDDING_BASE_URL`	向量服务地址（配置后启用向量腿）	空
`EMBEDDING_API_KEY`	向量服务密钥	空
`EMBEDDING_MODEL`	向量模型名	`BAAI/bge-m3`
`DATA_DIR`	SQLite 与上传文件目录	`./data`
`UPLOAD_DIR`	PDF 上传目录	`./data/uploads`
`MAX_UPLOAD_MB`	单文件上限	`120`
`MAX_PDF_PAGES`	单篇最大页数	`2000`
`PARSE_CONCURRENCY`	同时解析的论文数上限	`2`
`OCR_MAX_PAGES`	长扫描件最多 OCR 页数	`80`
`DOCLING_MAX_PAGES`	尝试 Docling 的最大页数	`60`
`FRONTEND_DIST`	前端构建产物目录	空

9.3 本地运行

```
cp .env.example .env
cd backend
python -m venv .venv
.venv\Scripts\activate
pip install -e ".[dev]"
uvicorn app.main:app --reload --port 8000
```

```
cd frontend
npm install
npm run dev
```

9.4 许可证

项目代码采用 Apache-2.0，Copyright 2026 niuchuanbo。第三方组件与本地开发技能声明见 `THIRD_PARTY_NOTICES.md`。模型权重与 API 服务遵循各自许可证与使用条款。

> 本文档描述的是**当前可运行版本**的真实实现与实测结果。未实现的能力在“已知降级项”和“局限”中明确列出，不做超出代码的宣称。